


Department of
Software Engineering

# Title: Introduction to assembly language and EMU 8086 instruction set

Microprocessors and microcontrollers

CSE 304




Green University of Bangladesh

# 1 Objective(s)

- Understand machine language, assembly language, instructions, assemblers and EMU8086.
- Identify the 8086 registers and explain 8-bit and 16-bit data.
- Read hexadecimal machine bytes, opcodes and register-encoding bits.
- Determine instruction sizes and observe `MOV` and `ADD` with Single Step.

# 2 Problem Analysis

Computers execute *machine instructions*, which are encoded in binary. Assembly language expresses processor operations with readable mnemonics, such as `MOV` and `ADD`. An **assembler** converts them into machine-code bytes. **EMU8086** allows us to execute those bytes and inspect registers and memory step by step.

| Assembly source ⟶ Assembler ⟶ Machine code ⟶ CPU execution |
|---|

**Why study assembly?**

Assembly helps us understand CPU execution, register use, instruction encoding, memory access and low-level debugging. It is relevant to computer architecture, embedded systems and microprocessor programming. **Basic terminology**

| | |
|---|---|
| Instruction | A processor-defined operation, such as data transfer or addition. |
| Mnemonic | A readable name, such as `MOV`, `ADD` or `RET`. |
| Operand | A register, immediate number or memory location used by an instruction. |
| Opcode | The machine-code field identifying a particular instruction form. |
| Assembler directive | A command for the assembler, not the CPU; example: `ORG`. |

**Bit, byte and word**

1 bit = 0 or 1; **8 bits = 1 byte**; **16 bits = 1 word** on 8086. One hexadecimal digit represents 4 bits. For example,

$$1011\,1000_2 = \mathrm{B8}_{16} = 184_{10}.$$

In EMU8086, a memory row showing `B8 184` displays **the same byte** in hexadecimal and decimal, not two separate instructions.

## 2.1 Architecture of EMU8086

The 8086 has 16-bit registers and a 20-bit physical address space (1 MiB). Instructions may occupy different numbers of bytes: a 16-bit CPU does *not* mean all instructions are 2 bytes long.

### 2.1.1 Central Processing Unit

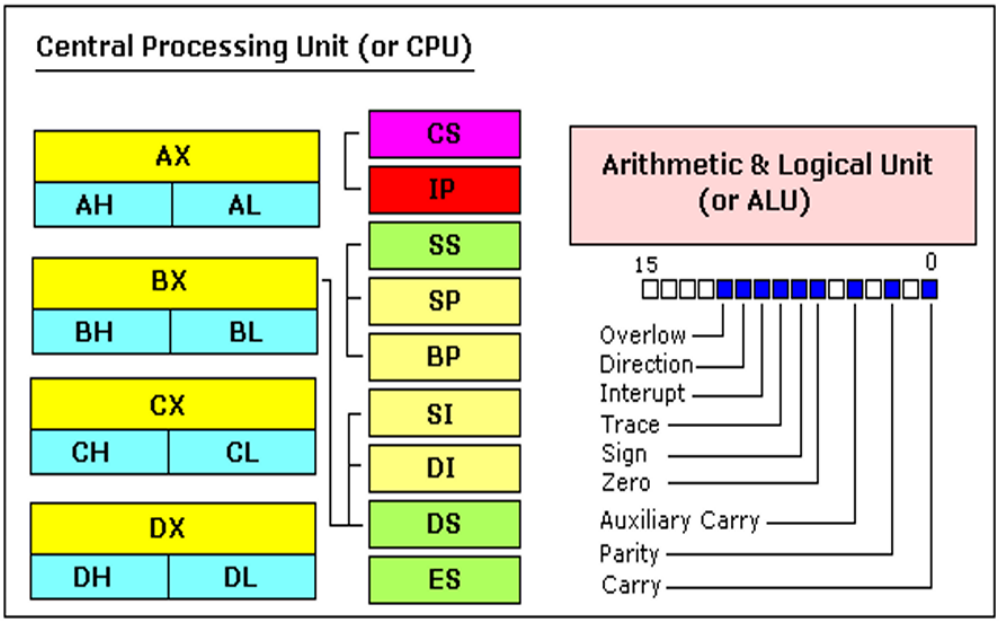



Figure 3: Central Processing Unit of EMU8086

Figure 1: Registers and status flags of the 8086 (from the supplied Lab 01 manual).

### 2.1.2 General Purpose Registers

| Register | Size | Typical use |
|---|---|---|
| AX | 16 bits | Accumulator; arithmetic and data processing. |
| BX | 16 bits | Base register; data and addressing. |
| CX | 16 bits | Counter; loops and shifts. |
| DX | 16 bits | Data register; I/O and extended arithmetic. |

Each of these 16-bit registers has two 8-bit halves: `AX = AH:AL`, `BX = BH:BL`, `CX = CH:CL`, `DX = DH:DL`. If `AX = 1234H`, then `AH = 12H` and `AL = 34H`.

### 2.1.3 Segment Registers

| Register | Role |
|---|---|
| CS | Code segment; paired with IP to locate the next instruction. |
| DS | Data segment; default segment for most data references. |
| SS | Stack segment; used with SP and BP. |
| ES | Extra segment; used in certain memory/string operations. |

### 2.1.4 Pointers, Index Registers and Flags

`IP` points to the next instruction's offset; `SP` is the stack pointer; `BP`, `SI` and `DI` support addressing. All are 16 bits. The 16-bit `FLAGS` register includes condition flags such as ZF (zero), CF (carry), SF (sign) and OF (overflow). **Segment:offset addressing**

In real mode, the physical address is segment $\times$ 10H + offset. Thus `CS:IP = 0700:0100` gives physical address `07100H`. `CS:IP = 0100:0000` gives `01000H`.

## 2.2 Instruction Set of EMU8086

An assembly instruction normally has the form `MNEMONIC destination, source`. Examples: `MOV AX,5`; `ADD AX,BX`. Some instructions have no explicit operand, such as `RET`.

| Group | Mnemonics | Purpose |
|---|---|---|
| Data transfer | MOV, XCHG, PUSH, POP | Transfer data. |
| Arithmetic | ADD, SUB, INC, DEC, MUL, DIV | Arithmetic. |
| Logical | AND, OR, XOR, NOT | Bitwise operations. |
| Control | JMP, CALL, RET | Change execution flow. |

## 2.3 Operand Types

| Type | Example | Meaning |
|---|---|---|
| Register | `MOV AX,BX` | Copy BX to AX. |
| Immediate | `MOV AX,5` | Constant 5 is inside instruction. |
| Memory | `MOV AL,[BX]` | Read from address DS:BX. |
| Memory direct | `MOV AX,[0120H]` | Read from DS:0120H. |

## 2.4 Data Transfer Instructions

| Instruction | Effect |
|---|---|
| `MOV AX,BX` | AX gets BX; BX remains unchanged. |
| `MOV AX,100H` | Put hexadecimal 0100 into AX. |
| `MOV CX,00F0H` | Put hexadecimal 00F0 (=240 decimal) into CX. |
| `MOV [BX],AL` | Store AL at memory address DS:BX. |

# 3 Example of MOV and ADD Command in EMU8086

```
org 100h
mov ax, 5
mov bx, 3
add ax, bx
ret
```

**Result:** AX = 0008H; BX = 0003H. This value appears in the *register pane*, not on the output screen. **What do ORG 100H and RET mean?**

`ORG 100H` is an assembler directive that sets the assumed starting *offset* to 0100H, used in DOS COM programs after the 256-byte Program Segment Prefix. It is *not* an executed instruction. `RET` is a CPU instruction (opcode `C3`) that returns from a procedure; in the simple COM example environment it can terminate via the DOS entry stack setup. Changing `ORG` alone does not relocate the loaded program in physical memory.

## 3.1 MOV and ADD: Opcode, Register Encoding and Instruction Size

**Today's focus:** four cases only: MOV immediate-to-register, MOV register-to-register, ADD immediate-to-register, and ADD register-to-register. Machine code is written in hexadecimal. Different valid encodings may represent the same operation. **Register codes (3 bits)**

| Bits | 16-bit | 8-bit |
|---|---|---|
| 000 | AX | AL |
| 001 | CX | CL |
| 010 | DX | DL |
| 011 | BX | BL |
| 100 | SP | AH |
| 101 | BP | CH |
| 110 | SI | DH |
| 111 | DI | BH |

The same three bits select different registers according to operand size. AX, BX, CX and DX are each 16 bits; their high and low halves (AH/AL, etc.) are 8 bits. Register numbers here follow the *encoding table*, not alphabetical order.

### Case 1: MOV constant to register (immediate)

**Example:** `MOV AX, 5`. Encoding: `10111rrr` followed by a 16-bit immediate (`imm16`). For AX, `rrr=000`, so the first byte is `10111000b = B8H`. The value 0005H appears as `05 00` (little-endian).

`MOV AX,5` ⟶ `B8 05 00` (3 bytes)

| Instruction | Machine bytes | Size |
|---|---|---|
| `MOV AX,5` | `B8 05 00` | 3 |
| `MOV BX,5` | `BB 05 00` | 3 |
| `MOV CX,5` | `B9 05 00` | 3 |
| `MOV DX,5` | `BA 05 00` | 3 |
| `MOV AX,1234H` | `B8 34 12` | 3 |
| `MOV AL,5` | `B0 05` | 2 |

**8-bit variation:** `MOV r8,imm8` uses `10110rrr` (B0–B7) followed by one immediate byte; for example `MOV AH,5` is `B4 05`.

### Case 2: MOV register to register

**Example:** `MOV AX,BX`. One valid encoding uses opcode `8B` (`MOV r16,r/m16`) plus a **Mod-R/M byte**. The second byte is divided into `mod | reg | r/m`.

`11 | 000 | 011 = 11000011b = C3H`

`mod=11`: source is a register; `reg=000`: destination AX; `r/m=011`: source BX.

`MOV AX,BX` $\longrightarrow$ `8B C3` (2 bytes)

| Instruction | Opcode | ModR/M | Complete bytes |
|---|---|---|---|
| `MOV AX,BX` | 8B | C3 | `8B C3` |
| `MOV AX,CX` | 8B | C1 | `8B C1` |
| `MOV BX,AX` | 8B | D8 | `8B D8` |
| `MOV CX,DX` | 8B | CA | `8B CA` |

**Alternative:** `MOV AX,BX` may also be encoded as `89 D8` (`MOV r/m16,r16`). Both are valid, two-byte encodings. Inspect the actual bytes selected by EMU8086.

### Case 3: ADD constant to register (immediate)

**Example:** `ADD AX,5`. Unlike a register-to-register ADD, the instruction contains the constant itself. The 8086 supports multiple legal immediate encodings:

| **Bytes** | **Form** | **Explanation** |
|---|---|---|
| `05 05 00` | `05 iw` | Special accumulator form: opcode 05, then 16-bit value 0005H |
| `83 C0 05` | `83 /0 ib` | General form: 83, ModR/M C0 for AX, sign-extended 8-bit constant 05 |
| `81 C0 05 00` | `81 /0 iw` | General form: 81, ModR/M C0 for AX, 16-bit constant 0005H |

Thus **there is no single opcode for every ADD-immediate instruction**. The assembler chooses an allowed form. The `/0` field means the ModR/M `reg` bits are `000`, selecting ADD; `mod=11`, `r/m=000` select AX. In the `83` encoding, the immediate is **signed 8-bit** (range $-128$ to 127), extended to 16 bits before addition.

| Instruction | One valid encoding | Size |
|---|---|---|
| `ADD AX,5` | `05 05 00` | 3 |
| `ADD BX,5` | `83 C3 05` | 3 |
| `ADD CX,5` | `83 C1 05` | 3 |
| `ADD AX,1234H` | `05 34 12` | 3 |

**8-bit accumulator:** `ADD AL,5` has the valid two-byte encoding `04 05`.

**Case 4: ADD register to register**

**Example:** `ADD AX,BX`. One encoding uses opcode `03` (`ADD r16,r/m16`), then ModR/M:

`mod=11 | reg=000 (AX) | r/m=011 (BX) ⇒ C3`

`ADD AX,BX` ⟶ `03 C3` (2 bytes)

| Instruction | Opcode | ModR/M | Complete bytes |
|---|---|---|---|
| `ADD AX,BX` | 03 | C3 | `03 C3` |
| `ADD AX,CX` | 03 | C1 | `03 C1` |
| `ADD BX,AX` | 03 | D8 | `03 D8` |
| `ADD CX,DX` | 03 | CA | `03 CA` |

**Alternative:** `ADD AX,BX` can also be `01 D8`. In both cases, AX receives AX+BX and BX is unchanged. ADD updates arithmetic status flags (e.g., ZF and CF), while MOV does not modify flags.

**Summary: four instruction forms**

| Case | Example | Machine bytes | Size |
|---|---|---|---|
| MOV immediate to register | `MOV AX,5` | `B8 05 00` | 3 |
| MOV register to register | `MOV AX,BX` | `8B C3` | 2 |
| ADD immediate to register | `ADD AX,5` | `05 05 00` | 3 |
| ADD register to register | `ADD AX,BX` | `03 C3` | 2 |

**Note:** The listed bytes show particular valid encodings. Some assemblers choose an equivalent form; compare the emulator's actual memory pane with the selected form. **How instruction sizes move the instruction pointer**

| COM offset | Machine bytes | Instruction |
|---|---|---|
| 0100–0102 | `B8 05 00` | `MOV AX,5` |
| 0103–0105 | `BB 03 00` | `MOV BX,3` |
| 0106–0107 | `03 C3` | `ADD AX,BX` |
| 0108 | `C3` | `RET` |

In this example the instruction sizes are 3, 3, 2 and 1 byte (total **9 bytes**). `ORG 100H` is a directive and has **no machine-code bytes**. The `C3` byte after `03` is a ModR/M byte; the separate `C3` at offset 0108 is the opcode for RET.

# 4 Input/Output

To observe the result, open EMU8086, create a COM program, compile/emulate it, then press **Single Step** three times. The AX register changes to `0005H` after the first step and `0008H`

after `ADD`. The memory pane displays machine bytes, while the register pane displays actual register values.

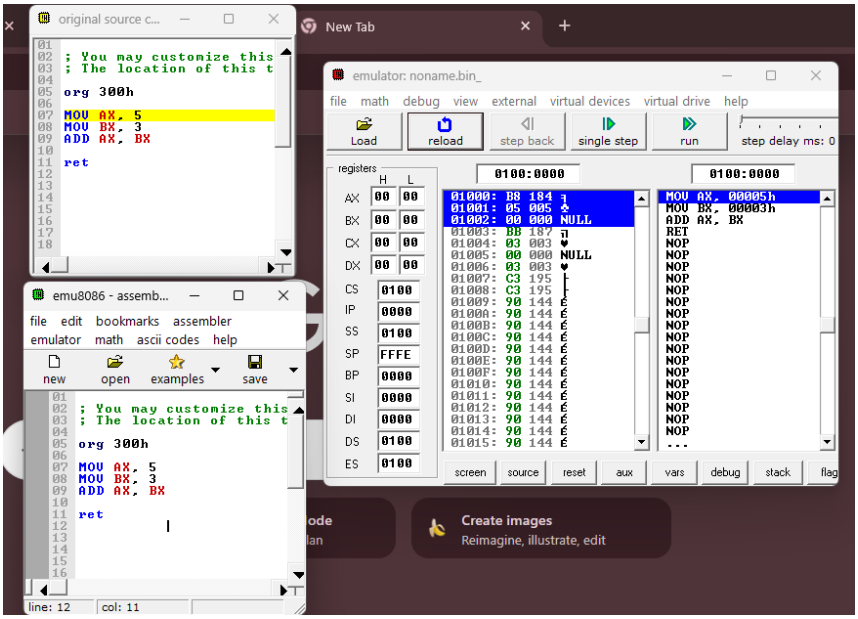



Figure 2: EMU8086 source editor and emulator showing machine-code bytes and instructions.

# 5 Discussion & Conclusion

A processor executes machine-code bytes, not the written mnemonic directly. Each instruction form has a defined encoding and possibly different size. The assembler creates these bytes; the emulator shows how they execute and change registers.

# 6 Lab Task (Please Implement Yourself and Show Output to the Instructor)

1. Open EMU8086, compile the sample COM program and observe the AX and BX registers using Single Step.
2. Execute `MOV AX,40H` and `ADD AX,10H`; report the final AX value.
3. Compare the machine bytes and sizes for all four forms: `MOV AX,5`, `MOV AX,BX`, `ADD AX,5`, and `ADD AX,BX`.

# 7 Lab Exercise (Submit as a Report)

1. Discuss the advantages and disadvantages of assembly language compared with high-level languages.
2. Put `100H` in BX, copy BX to AX and add `10H` to AX. Show machine bytes and final AX. (Expected: `0110H`.)

3. Explain why `B8` and `184` can appear side by side in the emulator memory window, and derive the encoding of `ADD AX,BX`.

# 8 Policy

Complete the tasks independently and provide your own program, emulator output and explanation. Copying another student's solution is prohibited.